

扑克牌花色识别系统

一、项目概要

1.需求说明

输入：斗地主游戏截图

输出：图内每一张扑克牌的包围框坐标、花色、牌面数字

2.分类规则

四种花色：黑桃 S、红桃 H、梅花 C、方块 D

牌面值：3、4、5、6、7、8、9、10、J、Q、K、A、2（13 种）

大小王 2 类：小王 BLACKJOK、大王 REDJOK

二、模型选择

YOLOv11m

三、数据集构建

本项目采用背景素材预处理 + 程序自动合成卡牌图像的方式构建大规模斗地主卡牌数据集，无需人工大量拍摄、标注，可快速生成上万张多样化训练样本，解决实拍图片数量不足、场景单一、标注成本高的问题。数据集共划分训练集、验证集两部分，支持 YOLOv11m 模型 54 类扑克牌检测任务训练。

1.背景素材预处理

为模拟线上斗地主游戏界面、桌面手牌等真实场景，首先构建纯

净背景图库：

（1）素材来源：网络收集大量不同风格斗地主游戏截图、桌面实拍背景图，包含不同 UI 布局、明暗光线、桌面纹理、多色块界面等多种场景；

（2）背景清洗处理：使用豆包帮助去除原图中所有原有扑克牌区域，擦除手牌、底牌、地主牌等卡牌元素，仅保留游戏界面边框、按钮、桌面底色、装饰纹理等无卡牌纯净背景；

（3）最终满足要求的背景图片放在 **background** 文件夹下面。

2.字体配置

由于不同斗地主游戏，其游戏内部卡牌的字体和花色是不同的。因此，只选用一种字体进行卡牌的随机生成，会导致模型的泛化能力不足。但是由于系统字体中，有很多字体可能缺少相关的花色的符号，或者是字体过于浮夸，不满足真实情况的需要。故而，本项目中通过人工筛选的形式，从几百个字体的款式中进行筛选，最后筛选的结果放在 `selected_fonts.txt` 文件中。

3.生成训练样本

基于 **Python + PIL** 图像处理库编写自动化数据集生成脚本，以预处理后的空白背景为基底，随机叠加扑克牌生成带标注的训练图片，核心随机化策略如下：

（1）分散式卡牌生成脚本

背景随机抽取：每次合成从背景池中随机选取一张空白界面作为

底图；

卡牌数量随机：单张图片随机生成 1~30 张卡牌，模拟零散手牌、少量底牌分布；

渲染参数随机：随机字体、字号（12~38）、牌面数字与花色两行文字行距，每张卡牌底色灰度在 240~252 区间随机；

位置防碰撞逻辑：遍历随机坐标时校验卡牌矩形区域重叠，最多重试 120 次寻找无冲突摆放位置，保证基础样本遮挡程度较低；

自动生成 YOLO 标签：通过 textbbox 获取每张卡牌文字区域像素坐标，调用 xyxy2yolo 函数转换为归一化中心坐标、宽高，一行对应一张卡牌类别与框信息，同步保存图片与.txt 标签文件。

其具体实现参考 01-gen-img.py，具体效果如下：



（2）堆叠式卡牌生成脚本

线上斗地主手牌存在大量卡牌前后堆叠、局部遮挡场景，为提升 YOLOv11m 遮挡检测鲁棒性，新增堆叠卡牌生成逻辑，是数据集核心难例来源，关键设计如下：

分多组堆叠布局：单张图片随机生成 1~6 组独立卡牌堆叠，每

组堆叠包含 1~15 张卡牌；

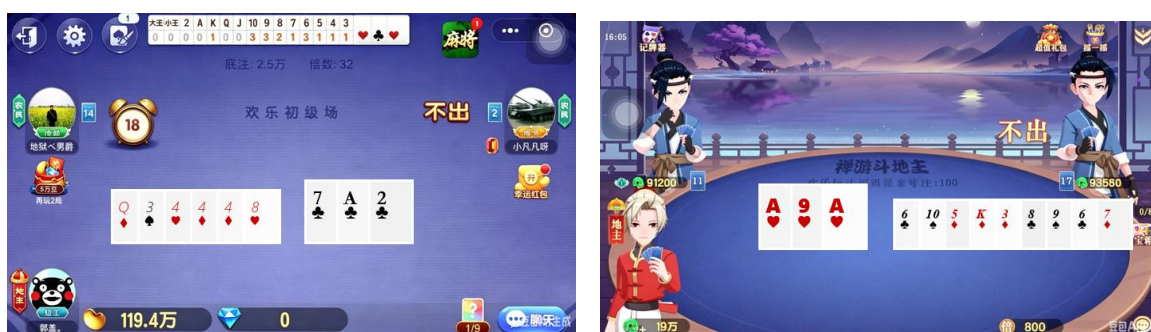
横向错位堆叠效果：同组内卡牌沿水平方向自适应偏移，偏移距离由卡牌宽度与偏移系数控制，模拟手牌层层遮挡视觉效果；

堆叠区域全局防冲突：以堆叠整体大矩形做碰撞检测，多组牌堆互不重叠，所有堆叠统一固定画面底部基线，贴合游戏手牌底部排布特征；

边界溢出保护：自动计算单堆叠整体宽度，超出画面可用宽度则跳过该堆叠，避免卡牌超出图片边界造成无效标注；

自动生成 YOLO 标签：通过 `textbbox` 获取每张卡牌文字区域像素坐标，调用 `xyxy2yolo` 函数转换为归一化中心坐标、宽高，一行对应一张卡牌类别与框信息，同步保存图片与.txt 标签文件。

其具体实现参考 `02-gen-playing-cards-02.py`，具体效果如下：



四、模型训练

1.训练数据集概况

本项目数据集全部由 Python+PIL 自动化脚本合成构建，包含分散式手牌样本与多层堆叠遮挡手牌样本，充分模拟线上斗地主游戏截

图场景，总样本规模共 22000 张。

训练集：20000 张图像，配套 20000 份 YOLO 格式标注文件，涵盖多组卡牌堆叠、局部遮挡、不同字体大小、明暗底色、随机排布等各类场景，用于网络权重迭代学习；具体数据放在 `images/train` 和 `labels/train` 文件夹下面。

验证集（测试集）：2000 张图像，配套 2000 份标注文件，全程不参与权重更新，每轮训练结束后自动在该数据集上计算精度、损失、混淆矩阵，用于评估模型泛化能力、判断过拟合。具体数据放在 `images/val` 和 `labels/val` 文件夹下面。

数据集严格遵循 YOLO 目录规范，通过 `train.yaml` 配置文件统一管理路径、类别信息，类别总数 `nc=54`，完整覆盖 4 花色 13 点数普通牌 + 大小王。

2.模型选型与训练环境

项目为 54 类多类别目标检测任务，需要同时完成卡牌定位与花色、点数分类，对比 YOLOv11n、YOLOv11s、YOLOv11m 三种轻量级模型，使用 YOLOv11m 模型，对细微花色、字符差异区分效果更强，兼顾检测精度与推理速度。

最终，对于 20000 张图片，训练 50 个 epoch，即得到最终的模型的权重。具体的训练代码放在 `train/train.py`；训练好后的模型权重保存在：`train/runs/detect/train-2/weights/best.pt`。

五、模型分析

针对上面得到的模型，对其用真实斗地主图片的数据进行测试。由于我们的验证集和测试集都是通过生成得到的，并不能很好的反应其在真实游戏场景中的效果。为此，本项目的测试是采用真实斗地主中的图片，但是由于我们并没有真实斗地主游戏图片的具体的标注数据，所以只能采用人工进行查看和修正。为快速进行测试，本项目采用 Annobel 自动标注工具对全部真实测试图片批量生成 YOLO 格式标注，自动框选画面内卡牌并匹配对应 54 类卡牌标签，少量自动标注偏差、漏框、错标样本人工修正，得到可用于客观评估模型泛化能力的真实场景测试集。

将训练完成的 YOLOv11m 最优权重 best.pt 导入推理程序，对全部真实斗地主图片批量推理检测，整体识别表现良好：画面中绝大多数完整、无重度遮挡的卡牌均可精准检出，卡牌包围框贴合卡牌实际边缘，不存在大幅偏移；无反光、光线均匀的清晰手牌，花色与牌点分类判断准确率高，黑桃、红桃、梅花、方块及大小王基本不会出现类别混淆；分散排布、无重叠遮挡的底牌、单张手牌识别稳定，置信度普遍维持在 0.85 以上；对于常规轻度重叠、仅小部分边缘遮挡的卡牌，模型仍能正常定位并正确区分卡牌类别，满足基础斗地主手牌识别需求。综合来看，模型在真实斗地主图片上具备基础可用的识别能力，大部分场景下能够完整输出画面内所有卡牌的包围框、花色与数字信息。

结合真实截图、实拍图的批量测试结果，模型在复杂真实环境中仍存在多处明显缺陷，核心问题分为误检、漏检、检测框不准三大类：

1.误检

上面得到的模型有时候会识别到比如只有花色的部分，可能会被识别到。具体如下：（下面图片中的单个方块 D 被误识别为某一个类别了，以及黑桃和上面的首出也被误识别）。



2.漏检

对于这类较小的卡牌，或者透明度有所改变的卡牌，可能会被漏检。



3.检测框不准

对于 JOKER 牌，在标注的过程中只标注了 JOK 三个字母，但是检测的时候总是把 JOKER 全部检测进去。



六、模型微调

经过真实斗地主图片测试后，模型存在单花色碎片误检、小尺寸、透明卡牌漏检、大王小王检测框偏移三类问题。结合问题根源，设计配套数据集扩充策略与微调训练方案，基于上面训练好的 `best.pt` 权重做二次微调，针对性修复上述缺陷。

1. 三类问题出现的原因

（1）单花色碎片误检问题：训练阶段合成数据集均为完整卡牌（数字 + 花色成对出现），不存在仅单独花色符号、无对应数字的局部图案样本。模型学习到花色特征权重过高，对画面中孤立的方块、黑桃小图标、界面残留花色碎片过度敏感，将局部花色图案判定为完整卡牌，产生大量误检框。

（2）小尺寸、低透明度卡牌漏检问题：原始合成脚本仅固定卡牌不透明底色，未模拟半透明、弱化显示的游戏手牌；同时样本内卡牌尺寸分布集中，缺少极小尺寸远距离卡牌样本。模型对低像素、透明度衰减、视觉特征弱化的卡牌特征提取能力不足，出现漏检。

（3）大小王 JOK 检测框范围不准问题：原合成脚本渲染大小王时仅绘制竖排 JOK 三段字符，但未区分完整 JOKER 长文本与标注目标区域；训练标签仅框选 JOK 核心字符，而图像中完整绘制了 JOKER 相关视觉区域，图像视觉范围与标注框范围不匹配，模型学习边界特征混乱，推理时自动放大检测框，框入全部 JOKER 区域，与标注标准不符。

2.微调数据集扩充与样本改造策略

新增定制化样本并修改原有渲染逻辑，生成补充样本参与微调训练，分别对应三类缺陷：

（1） 新增仅含孤立花色碎片样本，抑制花色过度敏感

修改 PIL 合成脚本，新增一类负干扰样本：仅在空白背景上绘制单独方块、黑桃、红桃、梅花花色符号，无配套数字字符，不赋予任何卡牌类别标签（作为纯负样本）。 将该类样本混入训练集，让模型区分「完整数字 + 花色卡牌」与「孤立花色碎片」，降低模型对单一花色图案的响应灵敏度，消除界面零散花色图标误检现象。

（2） 新增透明度随机扰动逻辑，解决透明卡牌漏检

优化卡牌渲染代码，对每张生成卡牌增加全局透明度随机调节参数，透明度区间设置 0.5~1.0 随机变化，模拟线上斗地主半透明手牌、弱化悬浮卡牌视觉效果。 扩充大量低透明度、小尺寸卡牌合成样本，丰富模型对弱视觉特征卡牌的特征学习，提升小目标、透明卡牌召回率，缓解漏检问题。

（3） 重构大小王渲染与标注逻辑，统一 JOK 框选标准

渲染层修改：大小王区域完整打印 JOKER 完整字符；

标注规则约束：生成标签时，仅截取、框选画面中 JOK 三段竖排核心字符作为标注目标，剩余 ER 字符不纳入标注框； 统一图像视觉内容与标注边界的对应关系，消除模型边界学习混淆，强制模型仅聚焦 JOK 核心区域输出检测框，解决检测框偏大、框入全部 JOKER 文字的问题。

具体的实现代码放在 `fine-tune/03-gen-img.py`

3. 模型微调

本次微调重新生成了 5000 张定制样本,对原来的 `best.pt` 进行微调,训练了 15 个 `epoch`。最终训练后得到的权重保存在 `fine-tune/train/runs/detect/train/weights/best.pt`。

4.模型分析

经过 5000 张定制样本、15 轮针对性微调后,模型在真实斗地主测试图片上整体识别效果提升显著,三类原有缺陷均得到有效改善:画面中单独散落的花色碎片基本不会再被误判为完整卡牌,游戏内尺寸偏小、带有透明度的手牌漏检情况大幅减少,大小王的检测框也能够精准贴合 JOK 核心文字区域,不会额外框入多余字符;整体来看各类完整卡牌定位、花色与点数分类识别稳定可靠,识别精度满足斗地主卡牌识别的使用需求,模型已具备落地使用的基础能力。

七、项目试错与迭代经历

1. 想法一

项目最开始采用两阶段识别思路,将数字、花色分开检测分类,仅设置 19 个类别用于训练,其中点数 13 类、4 种花色再叠加大小王;但落地测试后发现该方案存在根本性缺陷,单独识别出的花色与数字无法自动配对对应同一张卡牌,遇到卡牌重叠、画面多牌共存时极易出现匹配错乱,很难精准确定每张卡牌完整身份,无法直接输出单张牌的组合信息,后续业务解析逻辑复杂度极高,识别准确率难以

保障，因此直接推翻分拆识别的思路。

2. 想法二

第二种改进思路：采用两阶段流水线方案，第一阶段目标检测定位所有卡牌坐标，依靠检测框剔除背景 UI、零散花色等无关干扰，裁剪出单张卡牌小图后，第二阶段分类模型单独识别每张裁剪图的花色与点数。该逻辑理论上能规避背景干扰，但存在致命工程缺陷：整套流程需要两套独立数据集，不仅要标注卡牌定位框，还要对每张裁剪卡牌单独标注类别，全量人工标注工作量巨大，项目前期样本量需求上万，纯手动标注会耗费大量人力与时间，开发周期大，落地性价比极低，因此放弃双阶段分步识别方案。

3. 想法三

第三种方案计划直接截取某几款斗地主游戏内的单张卡牌截图作为素材，再基于截图素材生成训练数据集。该方法操作简单、素材获取直观，但存在致命缺陷：不同斗地主游戏 UI 的卡牌字体、花色图标尺寸、卡牌底色、配色风格差异极大，仅依靠固定几款游戏截图训练，模型只会适配有限样式的卡牌，特征学习单一，泛化能力薄弱，换到其他斗地主客户端截图、实体纸质扑克牌时会出现大量错分、漏检，无法实现通用卡牌识别，该素材采集方案也被推翻。

八、个人收获

通过本次斗地主卡牌识别项目的完整开发与迭代，我系统掌握了工程项目从需求分析、方案设计、问题排查到定向优化的完整解决思

路，深刻体会到深度学习模型开发并非一次性训练即可完成，而是一个持续发现问题、分析问题、针对性解决问题的迭代过程。在项目初期，我通过不断对比多种识别方案，逐步理解不同架构的优缺点，学会根据实际任务需求选择最合适的模型方案，摒弃了盲目训练模型的开发思维。

在模型训练与真实场景测试过程中，我能够精准观察模型缺陷、定位问题根源，不再单纯依赖调参提升精度。例如针对模型单独识别花色碎片造成误检的问题，我通过分析得出模型过度学习花色特征、无法区分完整卡牌与局部图案的核心原因，并针对性地引入大量单花色负样本参与训练，让模型自主学习“仅有花色、无对应数字的图案不属于目标卡牌”，从而从数据集层面约束模型检测逻辑，有效抑制误检现象。

同时，针对小尺寸、透明卡牌漏检、大小王检测框不准等问题，我也逐一分析成因，通过修改数据生成逻辑、增加透明度扰动、统一JOK标注规范、针对性扩充难例样本、低学习率微调等方式逐项解决。整个过程让我学会了先定位问题本质，再设计对应优化策略的科研思维，不再盲目堆砌数据、盲目调参。

总体而言，本次项目让我熟练掌握了数据集构建、模型训练、缺陷分析、定向微调的整套深度学习落地流程，培养了严谨的问题排查能力与工程迭代思维，为后续复杂视觉检测任务的开发与优化积累了扎实的实践经验。